

École Pour l'Informatique et les Techniques Avancées – EPITA

Masters program

Course: Introduction to Network protocols and Architecture (INPA)

INPA Course Outline

Course schedule (tentative)

Date	No.	Topics	Duration (in hours)
(check Zeus platform for the date & time)	1	Primer, TCP/IP stack, TCP/IP stack operations, Class exercises	3
(check Zeus platform for the date & time)	2	IP address allocation & routing, Subnetting, ARP, Class exercises	3
(check Zeus platform for the date & time)	3	DHCP, Packet handling, Routing Protocols (RIP, OSPF), Class exercises	3
(check Zeus platform for the date & time)	4	TCP, UDP, ICMP, ON, Class exercise	3
(check Zeus platform for the date & time)	5	Internet layer protocols (IGP vs EGP, BGP), ACLs, Class exercises	3
(check Zeus platform for the date & time)	6	Application Layer protocols/Security (HTTP/S), Class exercise	3
(check Zeus platform for the date & time)	7	Application Layer protocols/Security (SMTP + PGP), DNS, Class exercises	3
(check Zeus platform for the date & time)	8	NAT, Class exercise, Review/Practice-session	3
Total			24 hours
(check Zeus platform for the date & time)	EXAM		1

GRADING criteria:

Class attendance and responsiveness:	10%
Class assignments (practical work):	40%
Final evaluation (Quiz & Exam):	50%

Lecture 6 Outline

▶ HTTP

- Client–server setting
- Stateless
- Advantages & limitations

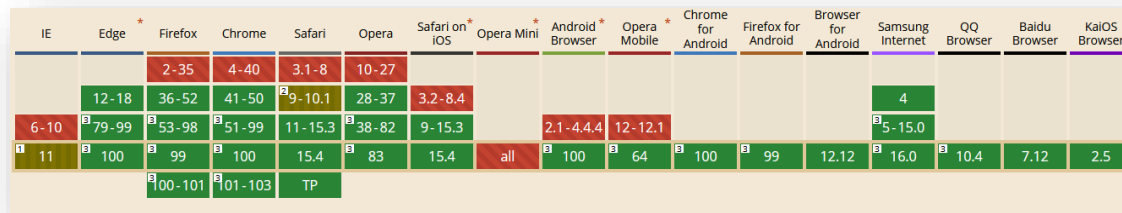
▶ HTTPS

- Port & structuring
- HTTP & SSL/TLS
- SSL/TLS in X509
- Limitations

Hypertext Transfer Protocol (HTTP)

(from TCP/IP 5 Layer model perspective)

- ▶ Introduction: Set of rules and standards that govern how information is transmitted on 'World Wide Web' (i.e., www)
 - Allow connected devices on 'www' to talk to each other
 - HTTP/1.1 was first documented in RFC 2068, and later replaced by newer RFCs (current: RFC 7230)
- ▶ HTTP/2 was standardized in RFC 7540:
 - Major improvements e.g., Low-latency transport, Multiplexing: single TCP connection for multiple requests, etc
 - Supported by most browsers



A screenshot of the 'Can I use' website showing HTTP/2 browser support. The table lists various browsers and their versions, with color-coded cells indicating support status: green for supported, red for not supported, and yellow for partial support. The browsers listed are IE, Edge, Firefox, Chrome, Safari, Opera, Safari on iOS, Opera Mini, Android Browser, Opera Mobile, Chrome for Android, Firefox for Android, Browser for Android, Samsung Internet, QQ Browser, Baldu Browser, and KaiOS Browser. The table shows that most modern browsers support HTTP/2, while older versions of IE and some mobile browsers do not.

IE	Edge	Firefox	Chrome	Safari	Opera	Safari on iOS	Opera Mini	Android Browser	Opera Mobile	Chrome for Android	Firefox for Android	Browser for Android	Samsung Internet	QQ Browser	Baidu Browser	KaiOS Browser
		2-35	4-40	3.1-8	10-27											
	12-18	36-52	41-50	9-10.1	28-37	3.2-8.4							4			
6-10	79-99	53-98	51-99	11-15.3	38-82	9-15.3		2.1-4.4.4	12-12.1				5-15.0			
11	100	99	100	15.4	83	15.4	all	100	64	100	99	12.12	16.0	10.4	7.12	2.5
		100-101	101-103	TP												

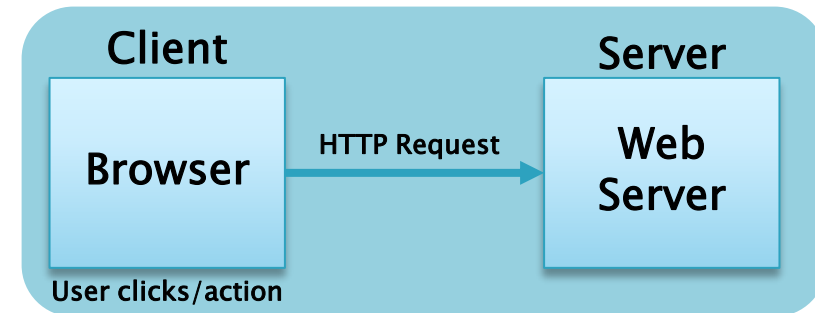
Screenshot from: Can I use - HTTP/2 browser support.
Retrieved from <https://caniuse.com/http2>

HTTP: Client-server architecture

- ▶ It is based on client-server architecture:
 - HTTP client (browser) sends a request to an HTTP server, which then send response back to the client

Request types:

GET, HEAD, POST, PUT, DELETE
TRACE, OPTIONS, CONNECT, PATCH, ...



- ▶ Popular HTTP Server implementations: Apache HTTP server, Microsoft IIS Jigsaw, Zope, ...
- ▶ Each client-server transaction, whether a request or a response, consists of three main parts
 - A response or request line
 - Header information
 - The body

```
GET / HTTP/2
Host: www.epita.fr
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:137.0) Gecko/20100101 Firefox/137.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br, zstd
Connection: keep-alive
Cookie: pll_language=en; __zlcmid=1R0nVvp2sNN12YA; _ga_XKKFXN2BFT=GS1.1.1743728969.1.1.1743728982.0.0.237607134;
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: none
Sec-Fetch-User: ?1
DNT: 1
Sec-GPC: 1
Priority: u=0, i
```

HTTP: Operation

- ▶ HTTP is stateless (lifetime of a connection corresponds to a single request–response sequence)
 - No ‘memory’ between client connections: HTTP server implementation treats every request as if it was a new one (no network overhead to create and maintain session)
 - HTTP pages are cached by browsers, ISP’s caching proxy and other (potential) intermediaries – to load pages faster
- ▶ An HTTP client opens a TCP/IP connection to the server via a socket, transmits request, then waits for a reply
 - Server listens on port 80 (alternative port: 8080)
 - Once the request–response sequence is completed, the socket is closed

HTTP Limitations

- ▶ **Security Concerns:**
 - **Privacy:** Eavesdropper can see the content
 - No encryption methods are used, hence is open to Man In The Middle (MITM) attacks
 - **Integrity:** Eavesdropper can alter the content
 - No data integrity checks are present – and therefore the content can be modified
 - **Authentication:** Eavesdropper can spoof connections
 - Authentication is sent in the clear – anyone who intercepts the request can sniff credentials over the channel
 - **Other attacks:** non-repudiation, replay, DDoS (*food for thought* –> <https://github.com/shekyan/slowhttptest>), etc.
- ▶ **Need state management techniques to maintain the information across multiple request–response cycles**

Lecture 7 Outline

▶ HTTP

- Client-server setting
- Stateless
- Advantages & limitations

▶ HTTPS

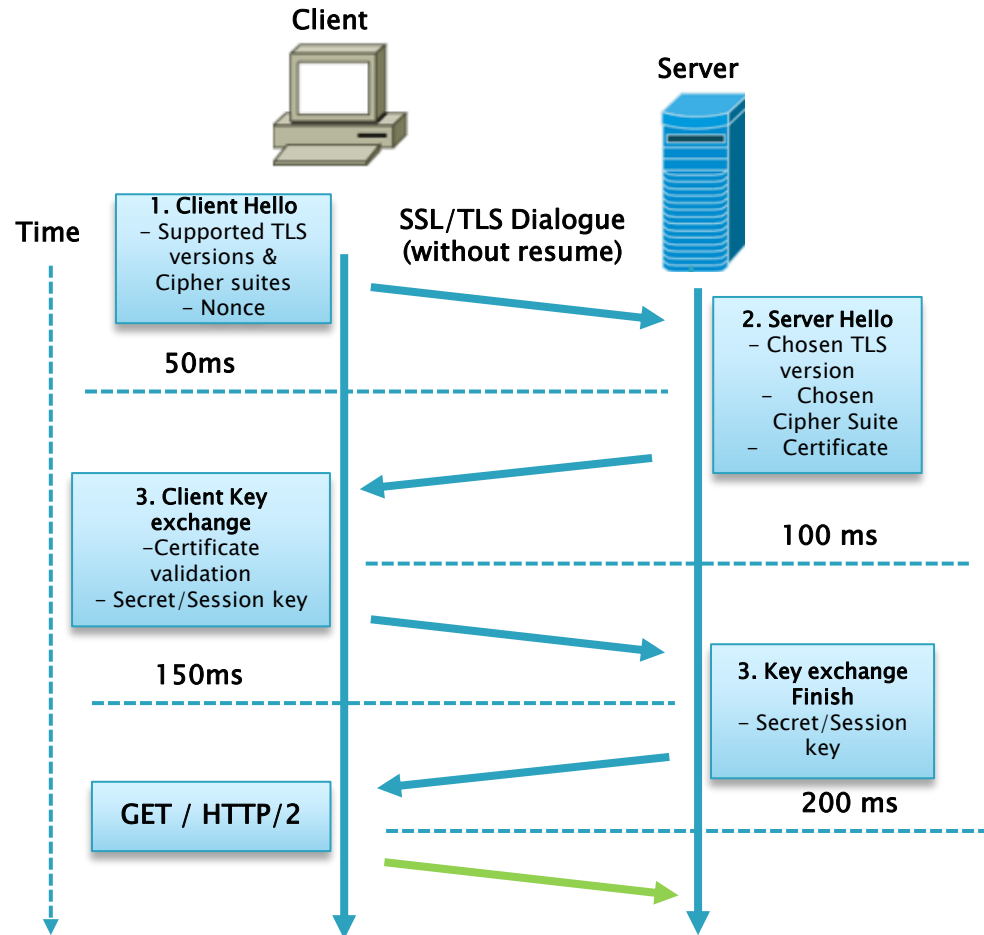
- **Port & structuring**
- **HTTP & SSL/TLS**
- **SSL/TLS in X509**
- **Limitations**

HTTP over SSL/TLS (HTTPS)

- ▶ SSL/TLS acts like another layer over regular HTTP
 - ▶ HTTPS encrypts an HTTP message prior to transmission and decrypts a message upon arrival
- ▶ Uses port 443 (as opposed to the standard HTTP port 80)
- ▶ URL's beginning with **HTTPS** indicate that the connection between client and browser is encrypted using SSL/TLS
 - ▶ E.g., `https://email.fr`
- ▶ SSL/TLS transactions are negotiated by means of a key based encryption algorithm between client and the server (typical RSA keypair sizes: 2048bits, 4096bits, ...)

SSL/TLS Handshake

- ▶ An HTTP-based SSL/TLS connection is always initiated by the client using a URL starting with <https://email.fr>
 - ▶ Requires a successful handshake in the beginning (simplified overview in the diagram)



SSL/TLS (in the context of X509)

- ▶ Need SSL/TLS if...
 - No if's and but's – use it whenever you can
- Certification Authority (CA) is an entity that issues digital certificates for use by other parties (it's an example of a trusted third party). E.g., VeriSign, Geotrust, etc
FREE: letsencrypt, StartSSL...
- ▶ Ability to connect to server via HTTPS consists of following sequential steps:
 - 1 – Generate key
 - 2 – Generate certificate signing request (CSR) to obtain CA signed certificate *OR* generate Self-signed certificate
 - 3 – Configure in web server

Digital certificate (X509)

When any modern browser is installed, it got sent with several CA issuer certificates. These issuer certificates contain a public key for the issuer, among other information

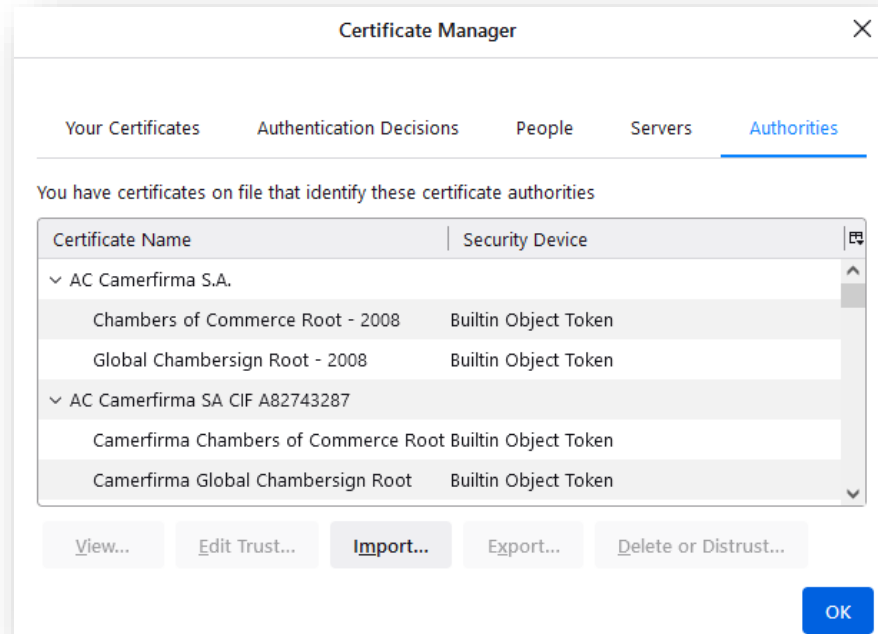
When a web admin decides to use SSL/TLS, a certificate should be purchased or obtained for free – that is signed using the CA's private key

The web browser starts a connection to an HTTP site. Along with this request the client sends all supported encryption schemes

As a response to the browser's connection request, the server sends a copy of the certificate from step2 to complete the cipher-suite negotiation

Once a certificate is received, the signature of the certificate (that was signed using the CA's private key) is checked using the CA's public key (installed in the browser in step 1)

The connection succeeds, the client can now communicate with the server using SSL/TLS



Screenshot from: Mozilla Foundation, Certificate Manager – Authorities tab (Firefox v102.0.1). Captured from locally installed browser, July 2022

SSL/TLS does not solve all issues

- ▶ Secure socket layer protocol protects and makes it easy for your Website visitors to trust you in three essential ways:
 - Confidentiality: an SSL/TLS certificate enables encryption of sensitive information
 - Integrity: Encryption ensures the prevention of any possible data modification
 - Authentication: each SSL/TLS certificate contains unique, authenticated information about the certificate owner
- ▶ Other issues?
 - SSL Stripping
 - Disclosure of domain name during TLS handshake, when host is serving multiple sites on the same IP address, via SNI extension and Encrypted Client Hello (ECH)
 - Mixed content e.g.,
 - <https://www.mixedcontentexamples.com/Test/NonSecureImage>
 - <https://googlesamples.github.io/web-fundamentals/fundamentals/security/prevent-mixed-content/active-mixed-content.html>

Lecture 6 ends here

- ▶ **Course Slides: Go to MS Teams:**
Introduction to Network protocols and architecture
→ Files section
- ▶ **Send your questions by email**
OR via direct message using MS Teams
- ▶ **Thank You!**